



Figure 4: Torch Demo (OFF)

```

        cam.setParameters(camParams);
        cam.startPreview();
    } catch (Exception e) {
        e.printStackTrace();
        cam.stopPreview();
        cam.release();
    }
}
});

```

The above code makes the camera flash operate as a torch. After you get the current parameters of the camera object, get its supported flash modes (which are generally [auto, on, off, torch]). Check if the array contains "FLASH_MODE_TORCH". If yes, then the torch mode is supported, otherwise an error dialogue is displayed. Once flash mode parameters are set, these are set for the camera object, and the camera preview is started to switch the flash on. The `stopPreview` call is to switch the flash off.

```

@Override
protected void onResume() {
    // TODO Auto-generated method stub
    super.onResume();
    if (cam == null) {
        cam = Camera.open();
    }
}

@Override
protected void onPause() {
    // TODO Auto-generated method stub
    super.onPause();
    if (cam != null) {
        cam.release();
    }
}

```

These life-cycle callback methods are implemented because we don't want to hold on to the camera object all the time; so release it as soon as the activity goes into a pause state, and regain control of the camera object to access it. `Cam.release()` is used to release control of the camera object.



Figure 5: Sensor list



Figure 6: The sensor that I have used for this demo app

```

@Override
protected void onStop() {
    // TODO Auto-generated method stub
    super.onStop();
    cam.release();
}
}

```

Accelerometer

Now that you know how to access phone hardware, it's time to look at some sensors. The most common of many sensors is the accelerometer. Let's build a simple app named 'Accelerometer Values' in which you can try and get the accelerometer values whenever you change the direction of your phone, and display acceleration in all (X, Y and Z) axes.

The accelerometer sensor is meant to simulate the acceleration forces like the speed changes that you see while playing games on phones. The values returned by the accelerometer for each axis range between (+/-)0-10. Let's look at how to design this app. With three axes in your co-ordinate system, you will need six text fields to display six different values for the (+/-) X, Y and Z axes.

```

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/
android"
    android:orientation="vertical" android:layout_width="fill_
parent"
    android:layout_height="fill_parent" android:layout_
gravity="center"
    android:background="@android:color/white">
    <TextView android:layout_width="match_parent"
    android:text="@string/x_axis"
        android:layout_height="wrap_content"
    android:gravity="center"

```